Chapter 2 From Hugging Face: Introduction to Q-Learning

06 May 2024 6:31

1. Introduction

In this unit, we're going to dive deeper into one of the Reinforcement Learning methods: value-based methods and study our first RL algorithm: Q-Learning. We'll also implement our first RL agent from scratch, a Q-Learning agent, and will train it in two environments:

a. Frozen-Lake-v1 (non-slippery version): where our agent will need to go from the starting state (S) to the goal state (G) by walking only on frozen tiles (F) and avoiding holes (H).
b. An autonomous taxi: where our agent will need to learn to navigate a city to transport its passengers from point A to point B.

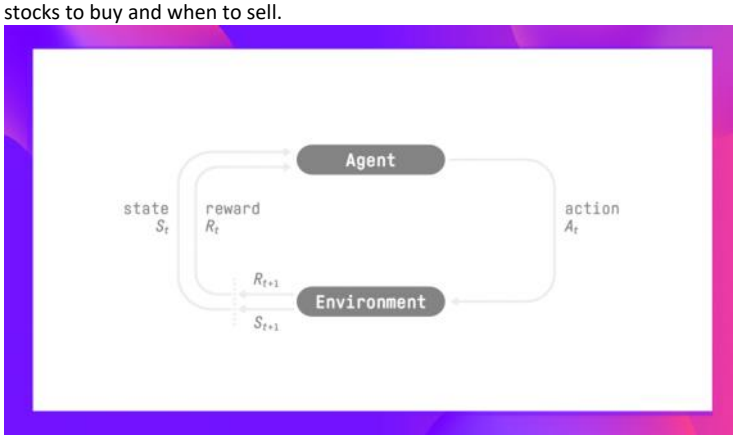
Concretely, we will:

- Learn about value-based methods.
- Learn about the differences between Monte Carlo and Temporal Difference Learning.
- Study and implement our first RL algorithm: Q-Learning.

This unit is fundamental if you want to be able to work on Deep Q-Learning: the first Deep RL algorithm that played Atari games and beat the human level on some of them (breakout, space invaders, etc)

2. Short Recap about Reinforcement Learning

In RL, we build an agent that can make smart decisions. For instance, an agent that learns to play a video game. Or a trading agent that learns to maximize its benefits by deciding on what stocks to buy and when to sell.



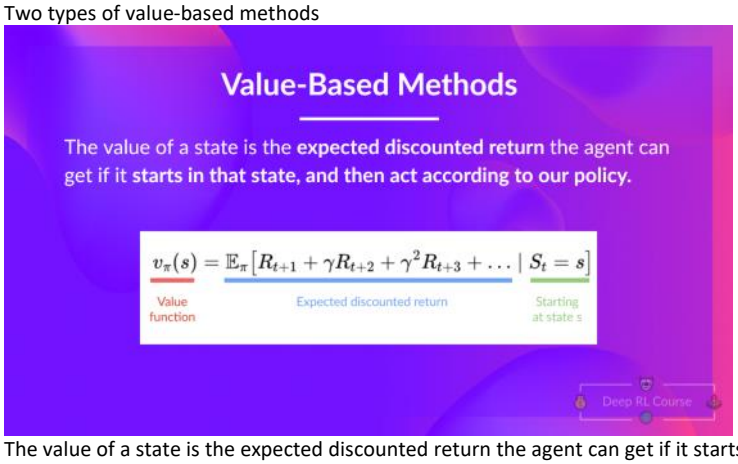
To make intelligent decisions, our agent will learn from the environment by interacting with it through trial and error and receiving rewards (positive or negative) as unique feedback. Its goal is to maximize its expected cumulative reward (because of the reward hypothesis). The agent's decision-making process is called the policy π: given a state, a policy will output an action or a probability distribution over actions. That is, given an observation of the environment, a policy will provide an action (or multiple probabilities for each action) that the agent should take.

Our goal is to find an optimal policy π* , aka., a policy that leads to the best expected cumulative reward.
And to find this optimal policy (hence solving the RL problem), there are two main types of RL methods:

- *Policy-based methods*: Train the policy directly to learn which action to take given a state.
- *Value-based methods*: Train a value function to learn which state is more valuable and use this value function to take the action that leads to it.

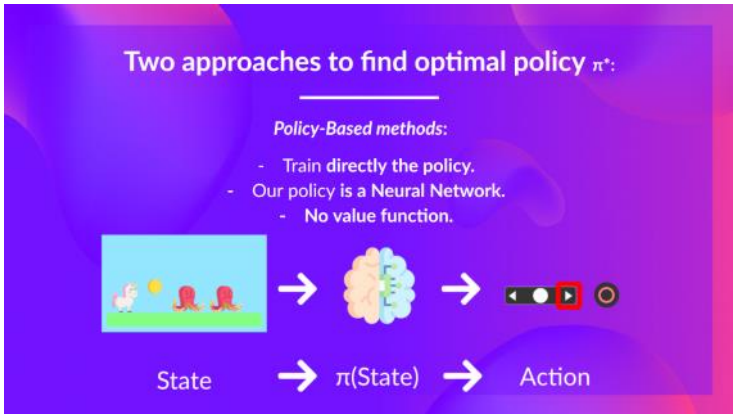
And in this unit, we'll dive deeper into the value-based methods.

3. Two types of value-based methods



The value of a state is the expected discounted return the agent can get if it starts at that state and then acts according to our policy. But what does it mean to act according to our policy? After all, we don't have a policy in value-based methods since we train a value function and not a policy. Remember that the goal of an RL agent is to have an optimal policy π*. To find the optimal policy, we learned about two different methods:

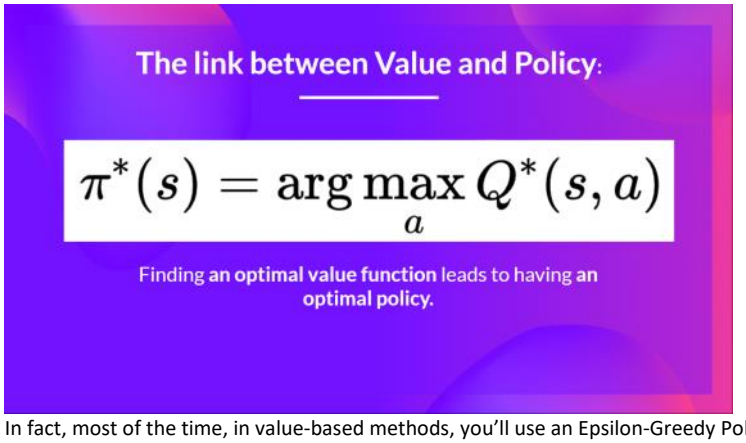
a. Policy-based methods: Directly train the policy to select what action to take given a state (or a probability distribution over actions at that state). In this case, we don't have a value function.
   The policy takes a state as input and outputs what action to take at that state (deterministic policy: a policy that output one action given a state, contrary to stochastic policy that output a probability distribution over actions). And consequently, we don't define by hand the behavior of our policy; it's the training that will define it.



b. Value-based methods: Indirectly, by training a value function that outputs the value of a state or a state-action pair. Given this value function, our policy will take an action. Since the policy is not trained/learned, we need to specify its behavior. For instance, if we want a policy that, given the value function, will take actions that always lead to the biggest reward, we'll create a Greedy Policy.

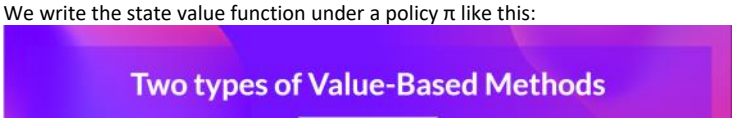
Given a state, our action-value function (that we train) outputs the value of each action at that state. Then, our pre-defined Greedy Policy selects the action that will yield the highest value given a state or a state action pair.
Consequently, whatever method you use to solve your problem, you will have a policy. In the case of value-based methods, you don't train the policy: your policy is just a simple pre-specified function (for instance, the Greedy Policy) that uses the values given by the value-function to select its actions.
So the difference is:

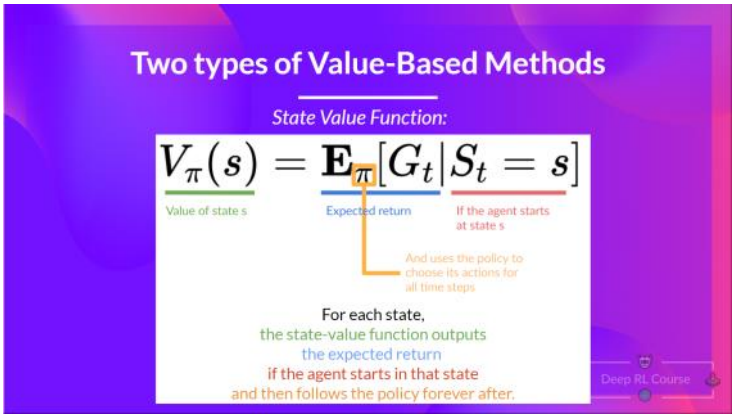
- In policy-based training, the optimal policy (denoted π*) is found by training the policy directly.
- In value-based training, finding an optimal value function (denoted Q* or V*, we'll study the difference below) leads to having an optimal policy.



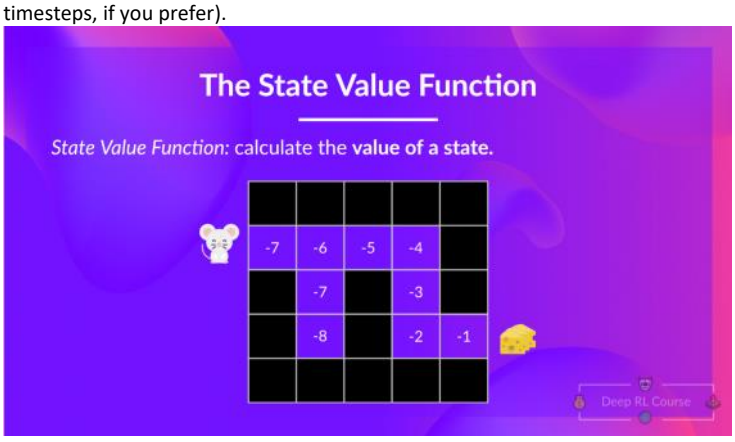
In fact, most of the time, in value-based methods, you'll use an Epsilon-Greedy Policy that handles the exploration/exploitation trade-off; we'll talk about this when we talk about Q-Learning in the second part of this unit. As we mentioned above, we have two types of value-based functions:

i. The state-value function
We write the state value function under a policy π like this:

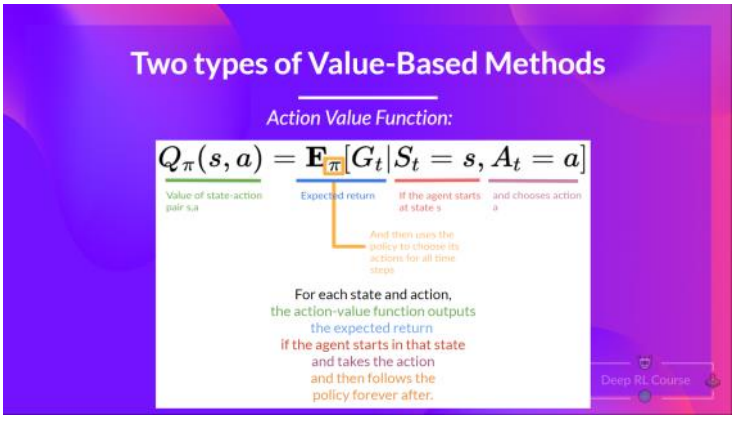


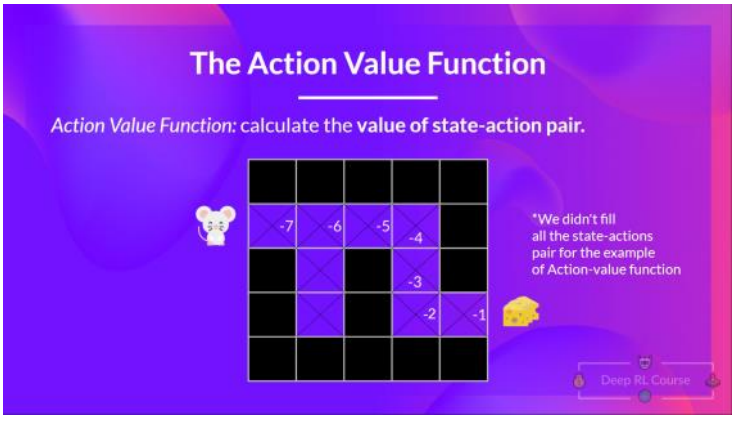


For each state, the state-value function outputs the expected return if the agent starts at that state and then follows the policy forever afterward (for all future timesteps, if you prefer).



ii. The action-value function
In the action-value function, for each state and action pair, the action-value function outputs the expected return if the agent starts in that state, takes that action, and then follows the policy forever after. The value of taking action a in states s under a policy π is:

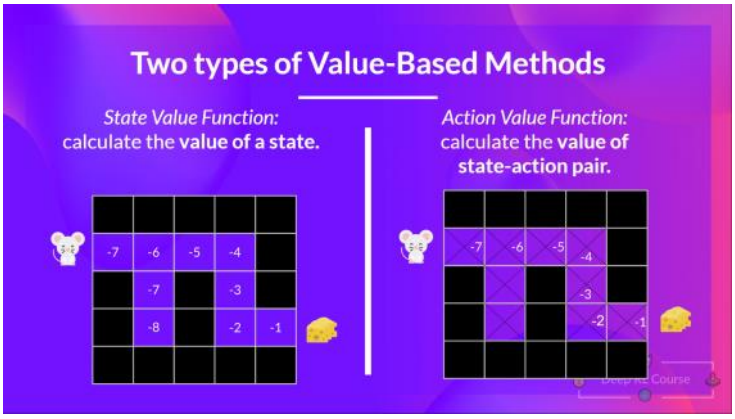




We see that the difference is:

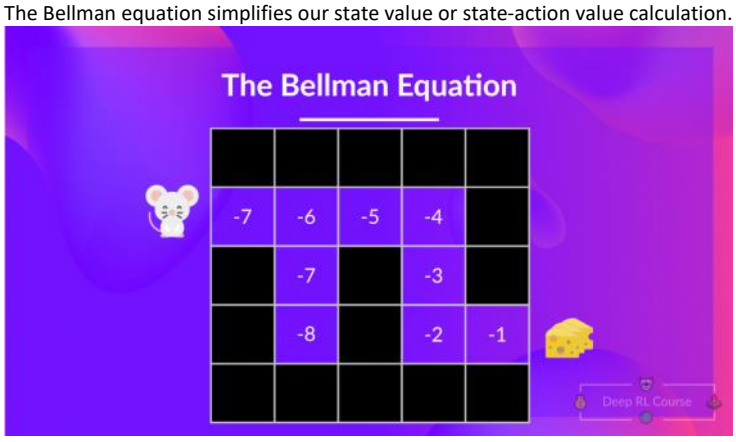
- For the state-value function, we calculate the value of a stateStSt

- For the action-value function, we calculate the value of the state-action pair (St,AtSt,At) hence the value of taking that action at that state.

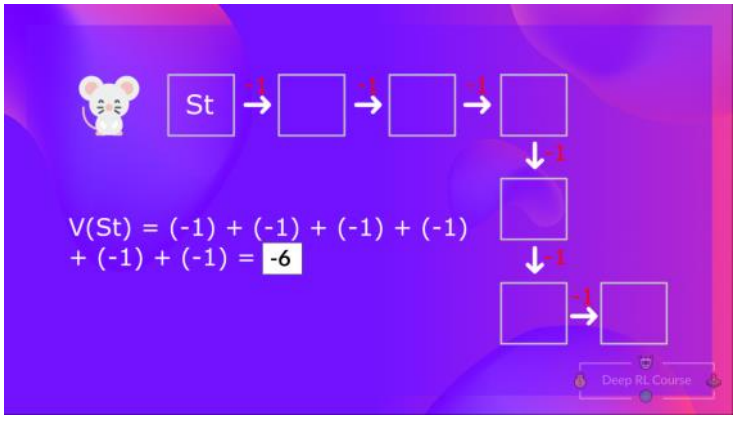


In either case, whichever value function we choose (state-value or action-value function), the returned value is the expected return.
However, the problem is that to calculate EACH value of a state or a state-action pair, we need to sum all the rewards an agent can get if it starts at that state.
This can be a computationally expensive process, and that's where the Bellman equation comes in to help us.

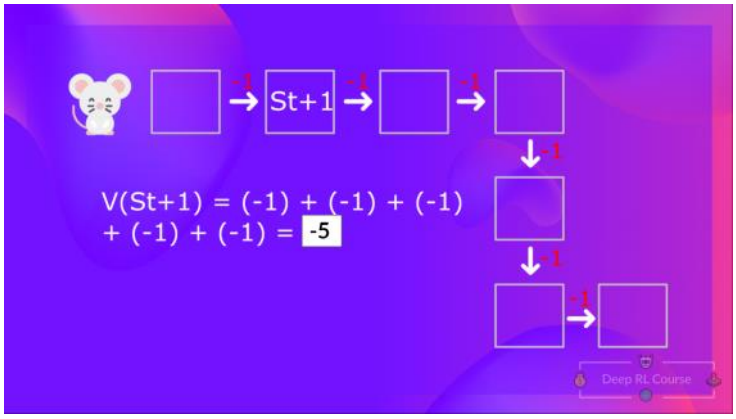
4. The Bellman Equation: simplify our value estimation
The Bellman equation simplifies our state value or state-action value calculation.



With what we have learned so far, we know that if we calculateV(St)V(St) (the value of a state), we need to calculate the return starting at that state and then follow the policy forever after. (The policy we defined in the following example is a Greedy Policy; for simplification, we don't discount the reward). So to calculateV(St)V(St), we need to calculate the sum of the expected rewards. Hence:



To calculate the value of State 1: the sum of rewards if the agent started in that state and then followed the greedy policy (taking actions that leads to the best states values) for all the time steps. Then, to calculate the$V(S_{t+1})$, we need to calculate the return starting at that state $S(_{t+1})$.

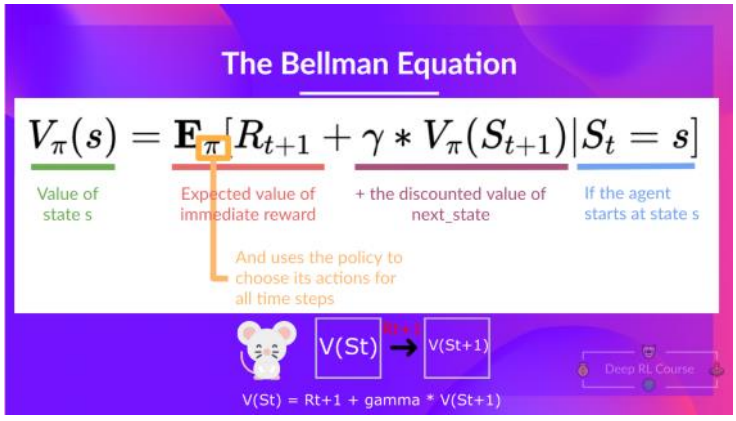


So you may have noticed, we're repeating the computation of the value of different states, which can be tedious if you need to do it for each state value or state-action value.

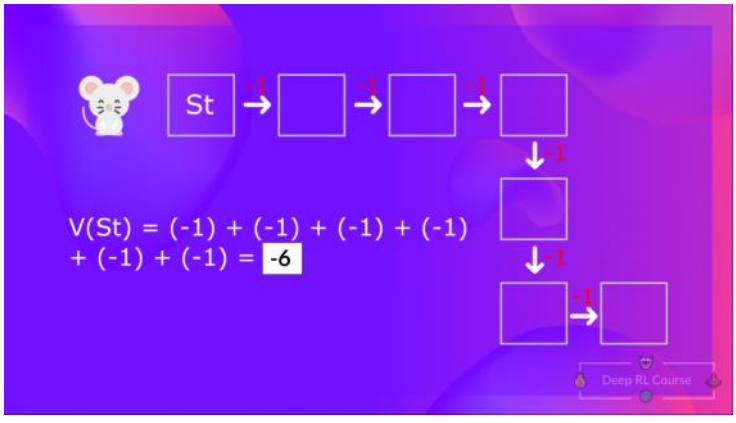
Instead of calculating the expected return for each state or each state-action pair, we can use the Bellman equation. (hint: if you know what Dynamic Programming is, this is very similar! if you don't know what it is, no worries!)

The Bellman equation is a recursive equation that works like this: instead of starting for each state from the beginning and calculating the return, we can consider the value of any state as:

The immediate reward R_{t+1} the discounted value of the state that follows ($\gamma * V(S_{t+1})$) .



If we go back to our example, we can say that the value of State 1 is equal to the expected cumulative return if we start at that state.



In the interest of simplicity, here we don't discount, so gamma = 1. But you'll study an example with gamma = 0.99 in the Q-Learning section of this unit.

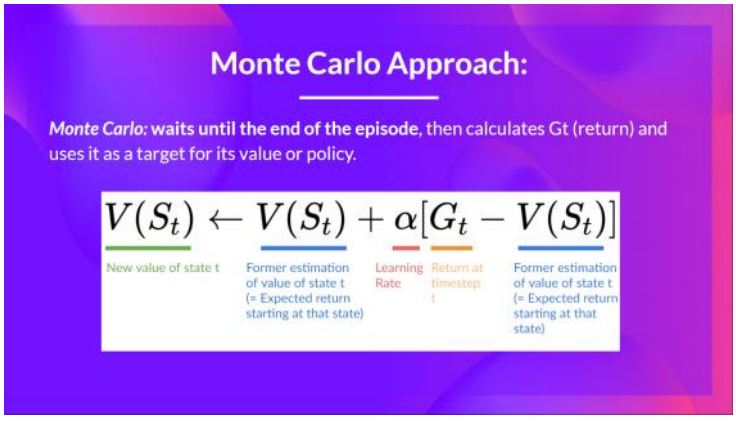
- The value of $V(S_{t+1})$= Immediate reward R_{t+2}+ Discounted value of the next state (gamma$*V(S_{t+2})$).
- And so on.

To recap, the idea of the Bellman equation is that instead of calculating each value as the sum of the expected return, which is a long process, we calculate the value as the sum of immediate reward + the discounted value of the state that follows.Before going to the next section, think about the role of gamma in the Bellman equation. What happens if the value of gamma is very low (e.g. 0.1 or even 0)? What happens if the value is 1? What happens if the value is very high, such as a million?

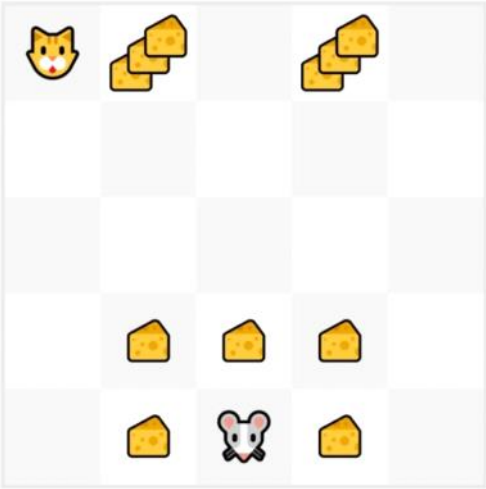
5. Monte Carlo vs Temporal Difference Learning

The last thing we need to discuss before diving into Q-Learning is the two learning strategies. Remember that an RL agent learns by interacting with its environment. The idea is that given the experience and the received reward, the agent will update its value function or policy. Monte Carlo and Temporal Difference Learning are two different strategies on how to train our value function or our policy function. Both of them use experience to solve the RL problem. On one hand, Monte Carlo uses an entire episode of experience before learning. On the other hand, Temporal Difference uses only a step (S_t,A_t,R_{t+1},S_{t+1}) to learn. We'll explain both of them using a value-based method example.

a. Monte Carlo: learning at the end of the episode
   Monte Carlo waits until the end of the episode, calculates GtGt (return) and uses it as a target for updating V(St)V(St). So it requires a complete episode of interaction before updating our value function.

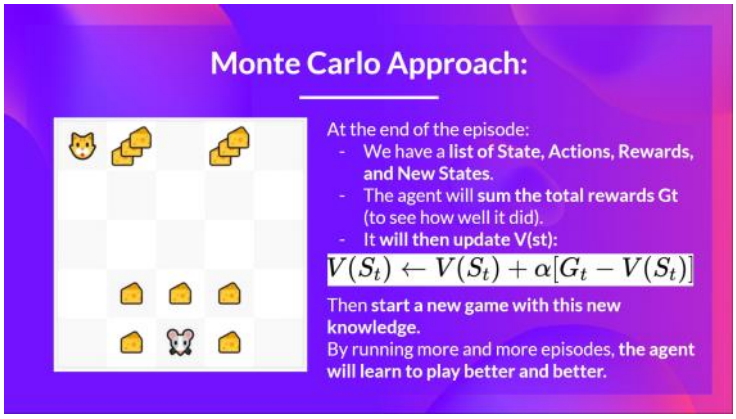


If we take example:



- We always start the episode at the same starting point.
- The agent takes actions using the policy. For instance, using an Epsilon Greedy Strategy, a policy that alternates between exploration (random actions) and exploitation.
- We get the reward and the next state.
- We terminate the episode if the cat eats the mouse or if the mouse moves > 10 steps.
- At the end of the episode, we have a list of State, Actions, Rewards, and Next States tuples For instance [[State tile 3 bottom, Go Left, +1, State tile 2 bottom], [State tile 2 bottom, Go Left, +0, State tile 1 bottom]…]
- The agent will sum the total rewards G_t (to see how well it did).
- It will then update$V(S_t)$based on the formula
- $$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$
  New value of state t; Former estimation of value of state t (= Expected return starting at that state); Learning Rate; Return at timestep t; Former estimation of value of state t (= Expected return starting at that state)
- Then start a new game with this new value

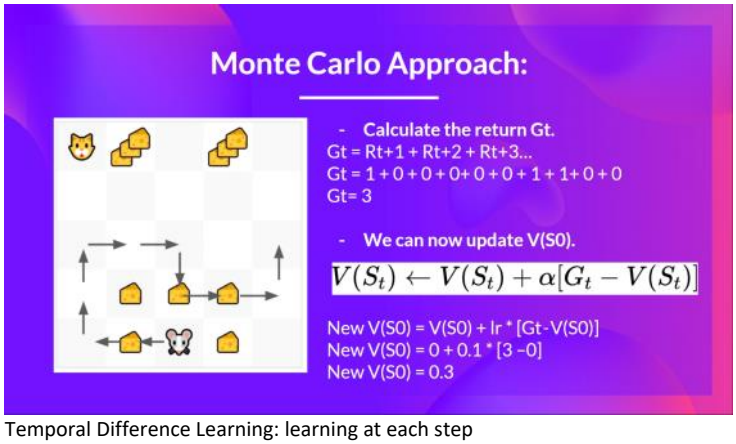
By running more and more episodes, the agent will learn to play better and better.



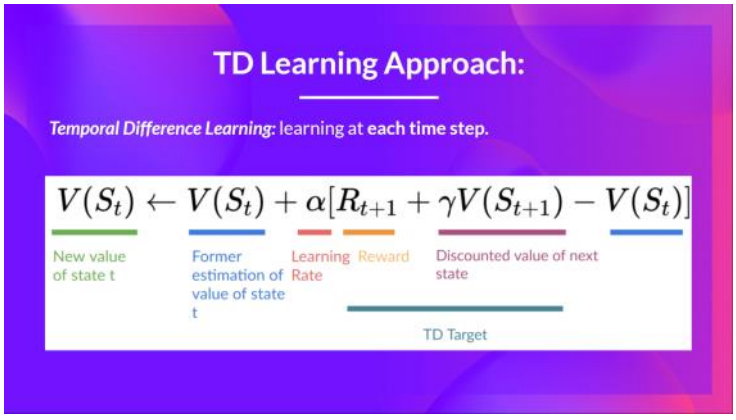
- We have a list of state, action, rewards, next_state, we need to calculate the return Gt=0, $G_t=R_{t+1}+R_{t+2}+R_{t+3}$.....(for simplicity, we don't discount the rewards) G0 =R1+R2+R3…G0=R1+R2+R3…G0=1+0+0+0+0+0+1+1+0+0G_0=3
- We can now compute the new $V(S_0)$:
- $$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

  New value of state t | Former estimation of value of state t (= Expected return starting at that state) | Learning Rate | Return at timestep t | Former estimation of value of state t (= Expected return starting at that state)

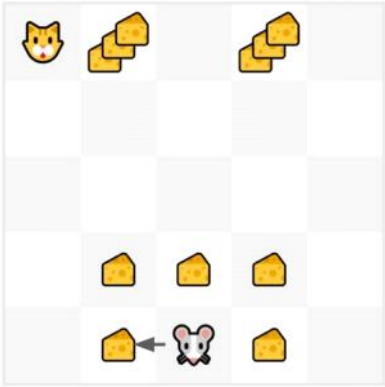
$V(S_0)=V(S_0)+lr*[G_0—V(S_0)]$ $V(S_0)=0+0.1*[3–0]V(S_0)=0+0.1*[3–0]$ $V(S_0)=0.3V(S_0)=0.3$



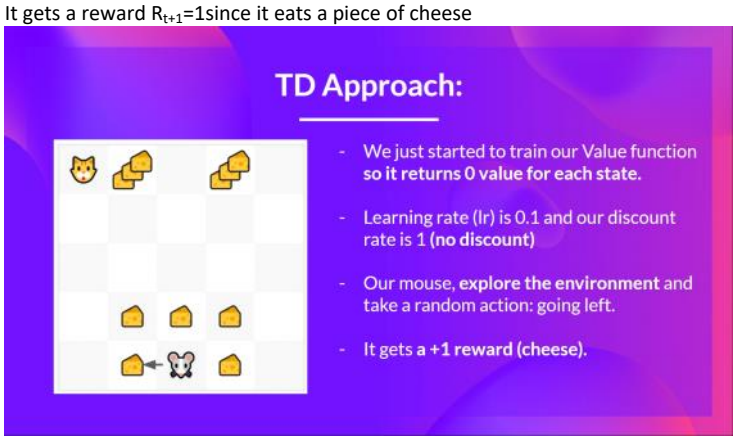
a. Temporal Difference Learning: learning at each step
   Temporal Difference, on the other hand, waits for only one interaction (one step)S_{t+1} to form a TD target and updateV(St) using R_{t+1} and $\gamma*V(St_{+1})$.
   The idea with TD is to update the$V(S_t)$ at each step.
   But because we didn't experience an entire episode, we don't haveGtGt(expected return). Instead, we estimate G_t by addingR_{t+1} and the discounted value of the next state.
   This is called bootstrapping. It's called this because TD bases its update in part on an existing estimate $V(S_{t+1})$ and not a complete sample Gt.



This method is called TD(0) or one-step TD (update the value function after any individual step).



- We initialize our value function so that it returns 0 value for each state.
- Our learning rate (lr) is 0.1, and our discount rate is 1 (no discount).
- Our mouse begins to explore the environment and takes a random action: going to the left
- It gets a reward R_{t+1}=1since it eats a piece of cheese



$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

New value of state t

Former estimation of value of state t

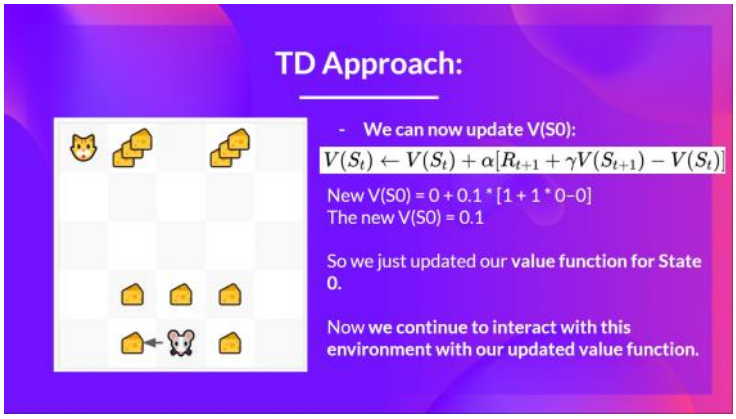
Learning Rate

Reward

Discounted value of next state

TD Target

We can now update $V(S_0)$:
New $V(S_0)=V(S_0)+lr*[R1+\gamma*V(S_1)-V(S_0)]$
$NewV(S_0)=0+0.1*[1+1*0-0]$
$NewV(S_0)=0.1$
So we just updated our value function for State 0. Now we continue to interact with this environment with our updated value function.



To summarize:

- With *Monte Carlo*, we update the value function from a complete episode, and so we use the actual accurate discounted return of this episode.
- With *TD Learning*, we update the value function from a step, and we replaceGtGt, which we don't know, with an estimated return called the TD target.

Monte Carlo: $V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$

TD Learning: $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

6. Mid-Way Recap

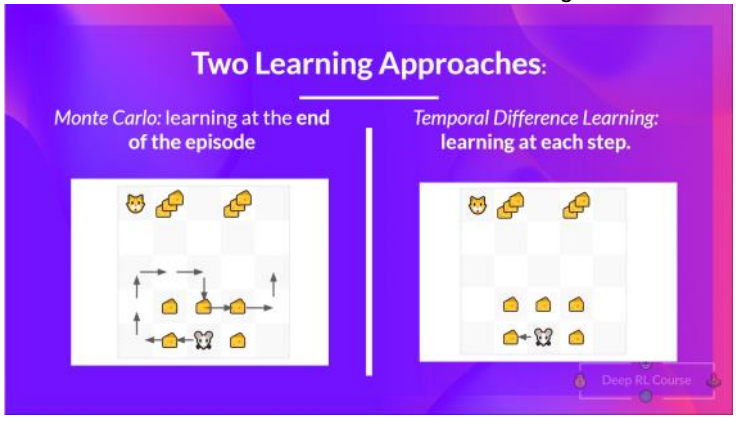
Before diving into Q-Learning, let's summarize what we've just learned.

We have two types of value-based functions:

- State-value function: outputs the expected return if **the agent starts at a given state and acts according to the policy forever after.**
- Action-value function: outputs the expected return if **the agent starts in a given state, takes a given action at that state** and then acts accordingly to the policy forever after.
- In value-based methods, rather than learning the policy, **we define the policy by hand** and we learn a value function. If we have an optimal value function, we **will have an optimal policy.**

There are two types of methods to update the value function:

- With *the Monte Carlo method*, we update the value function from a complete episode, and so we **use the actual discounted return of this episode.**
- With *the TD Learning method,* we update the value function from a step, replacing the unknown Gt with **an estimated return called the TD target.**



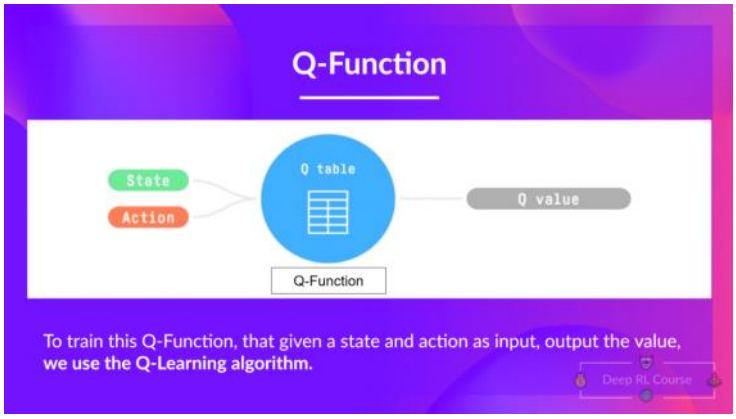
7. Introducing Q-Learning

   a. What is Q-Learning

   Q-Learning is an **off-policy value-based method that uses a TD approach to train its action-value function:**

   - *Off-policy*: we'll talk about that at the end of this unit.
   - *Value-based method*: finds the optimal policy indirectly by training a value or action-value function that will tell us **the value of each state or each state-action pair.**
   - *TD approach:* **updates its action-value function at each step instead of at the end of the episode.**

   Q-Learning is the algorithm we use to train our Q-function, an **action-value function** that determines the value of being at a particular state and taking a specific action at that state.

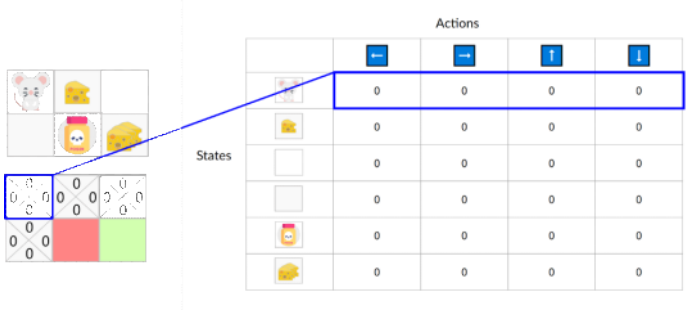


The **Q comes from "the Quality" (the value) of that action at that state.** Let's recap the difference between value and reward:

- The *value of a state*, or a *state-action pair* is the expected cumulative reward our agent gets if it starts at this state (or state-action pair) and then acts accordingly to its policy.
- The *reward* is the **feedback I get from the environment** after performing an action at a state.

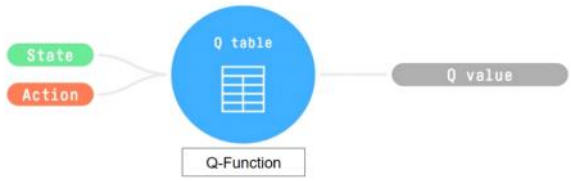
Internally, our Q-function is encoded by **a Q-table, a table where each cell corresponds to a state-action pair value.** Think of this Q-table as **the memory or cheat sheet of our Q-function.**

The Q-table is initialized. That's why all values are = 0. This table **contains, for each state and action, the corresponding state-action values.** For this simple example, the state is only defined by the position of the mouse. Therefore, we have 2*3 rows in our Q-table, one row for each possible position of the mouse. In more complex scenarios, the state could contain more information than the position of the actor.



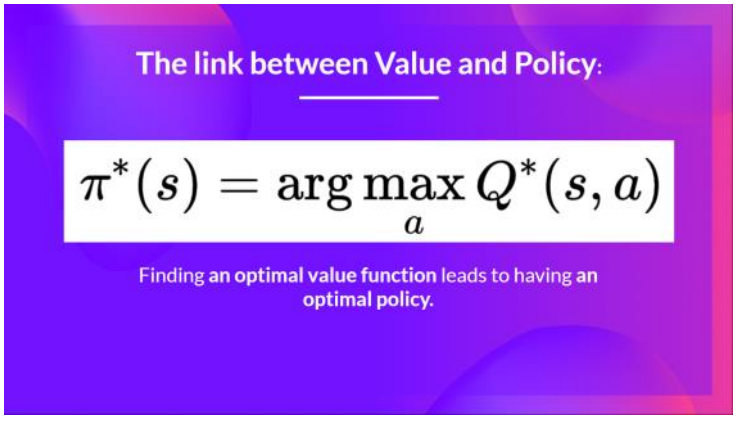
Here we see that the state-action value of the initial state and going up is 0:

So: the Q-function uses a Q-table **that has the value of each state-action pair.** Given a state and action, **our Q-function will search inside its Q-table to output the value.**

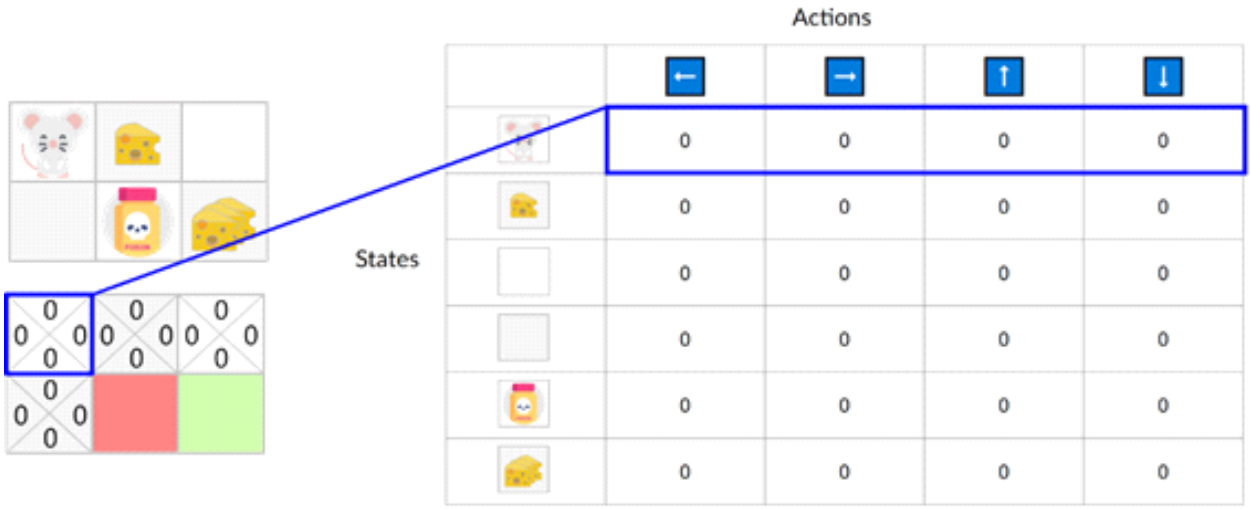


If we recap, *Q-Learning* **is the RL algorithm that:**

- Trains a *Q-function* (an **action-value function**), which internally is a **Q-table that contains all the state-action pair values.**
- Given a state and action, our Q-function **will search its Q-table for the corresponding value.**
- When the training is done, **we have an optimal Q-function, which means we have optimal Q-table.**
- And if we **have an optimal Q-function**, we **have an optimal policy** since we **know the best action to take at each state.**

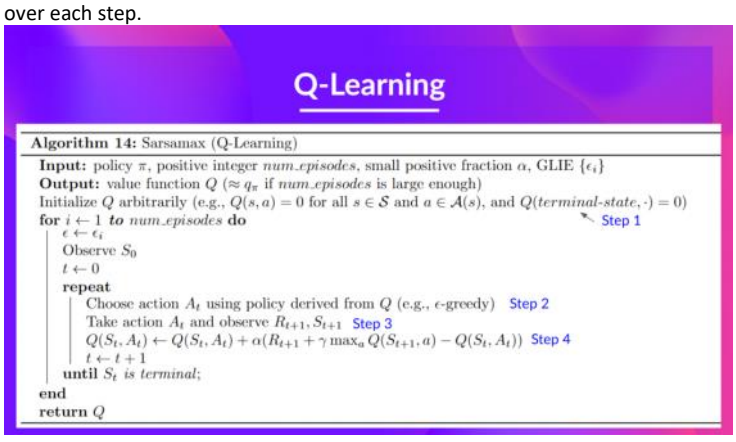


In the beginning, our Q-table is useless since it gives arbitrary values for each state-action pair (most of the time, we initialize the Q-table to 0). As the agent explores the environment and we update the Q-table, it will give us a better and better approximation to the optimal policy.

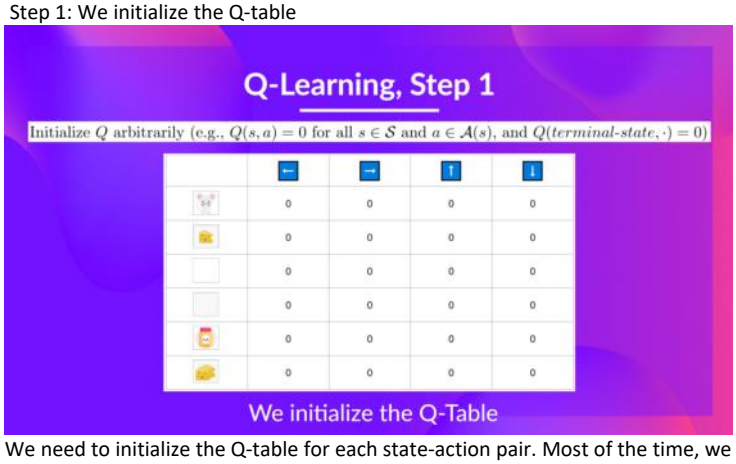


Now that we understand what Q-Learning, Q-functions, and Q-tables are, let's dive deeper into the Q-Learning algorithm

8. The Q-Learning algorithm
   This is the Q-Learning pseudocode; let's study each part and see how it works with a simple example before implementing it. Don't be intimidated by it, it's simpler than it looks! We'll go over each step.

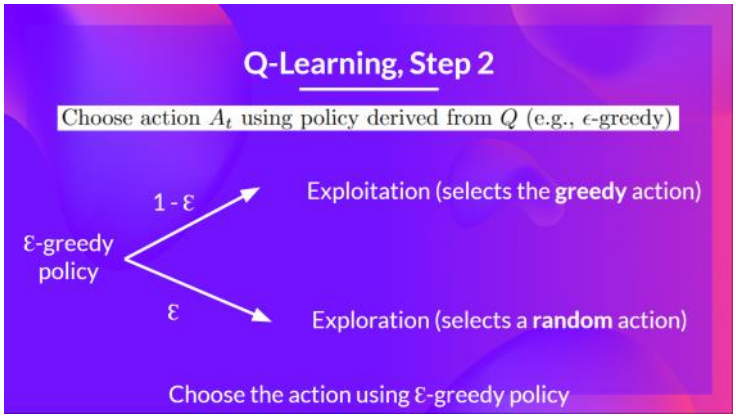


   a. Step 1: We initialize the Q-table



   We need to initialize the Q-table for each state-action pair. Most of the time, we initialize with values of 0.

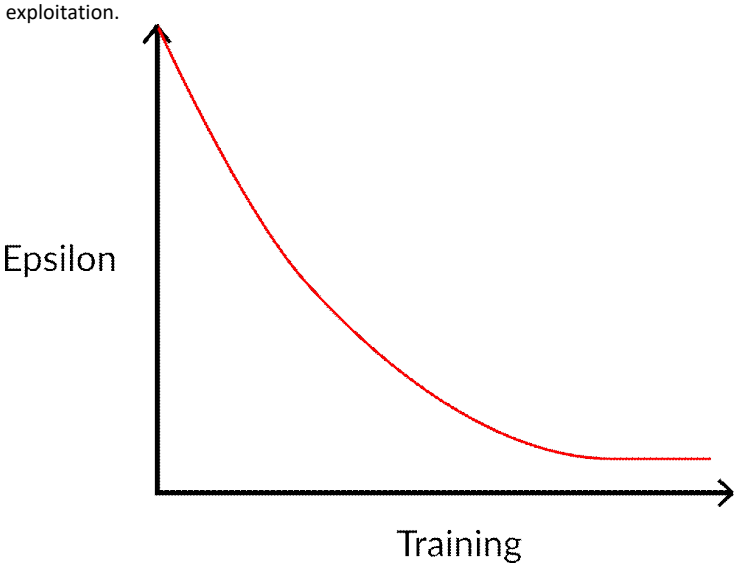
   b. Step 2: Choose an action using the epsilon-greedy strategy



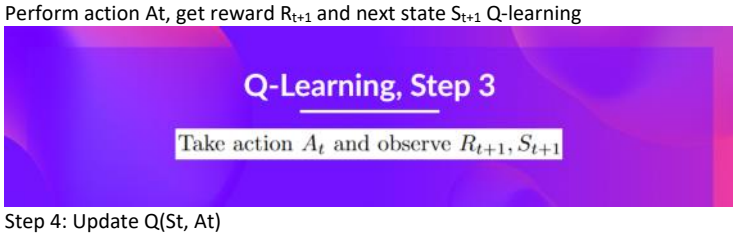
The epsilon-greedy strategy is a policy that handles the exploration/exploitation trade-off.
The idea is that, with an initial value of ε = 1.0:

- *With probability 1 — ε* : we do **exploitation** (aka our agent selects the action with the highest state-action pair value).
- With probability ε: **we do exploration** (trying random action).

At the beginning of the training, **the probability of doing exploration will be huge since ε is very high, so most of the time, we'll explore.** But as the training goes on, and consequently our **Q-table gets better and better in its estimations, we progressively reduce the epsilon value** since we will need less and less exploration and more exploitation.



c. Perform action At, get reward R_{t+1} and next state S_{t+1} Q-learning



d. Step 4: Update Q(St, At)
Remember that in TD Learning, we update our policy or value function (depending on the RL method we choose) **after one step of the interaction.** To produce our TD target, **we used the immediate reward R_{t+1} plus the discounted value of the next state**, computed by finding the action that maximizes the current Q-function at the next state. (We call that bootstrap).

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

New value of state t
Former estimation of value of state t
Learning Rate
Reward
Discounted value of next state
TD Target

Therefore, our Q(St,At) update formula goes like this:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

New Q-value estimation | Former Q-value estimation | Learning Rate | Immediate Reward | Discounted Estimate optimal Q-value of next state | Former Q-value estimation

TD Target

TD Error

This means that to update ourQ(St,At)Q(St,At):

- We need S_t,A_t,R_{t+1},S_{t+1}.
- To update our Q-value at a given state-action pair, we use the TD target.

How do we form the TD target?

1. We obtain the reward R_{t+1} after taking the actionA_t.
2. To get the **best state-action pair value** for the next state, we use a greedy policy to select the next best action. Note that this is not an epsilon-greedy policy, this will always take the action with the highest state-action value.

Then when the update of this Q-value is done, we start in a new state and select our action **using a epsilon-greedy policy again. This is why we say that Q Learning is an off-policy algorithm.**

9. Off-Policy and On-Policy

The difference is subtle:

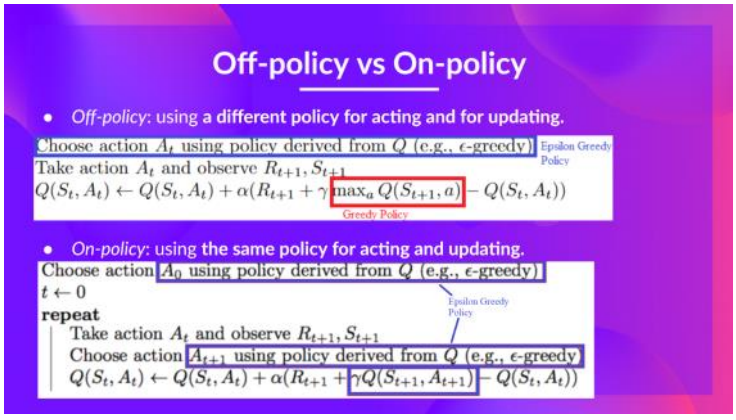
- *Off-policy*: using **a different policy for acting (inference) and updating (training).** For instance, with Q-Learning, the epsilon-greedy policy (acting policy), is different from the greedy policy that is **used to select the best next-state action value to update our Q-value (updating policy).**

Choose action A_t using policy derived from Q (e.g., ϵ-greedy)

- On-policy: using the same policy for acting and updating.
  For instance, with Sarsa, another value-based algorithm, the epsilon-greedy policy selects the next state-action pair, not a greedy policy

Choose action A_0 using policy derived from Q (e.g., ϵ-greedy)
$t \leftarrow 0$
repeat
  Take action A_t and observe R_{t+1}, S_{t+1}
  Choose action A_{t+1} using policy derived from Q (e.g., ϵ-greedy)
  $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$

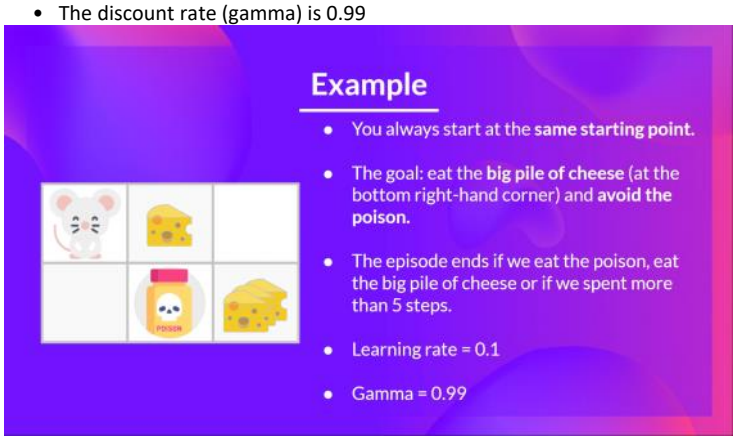
Epsilon Greedy Policy



10. A Q-Learning Example

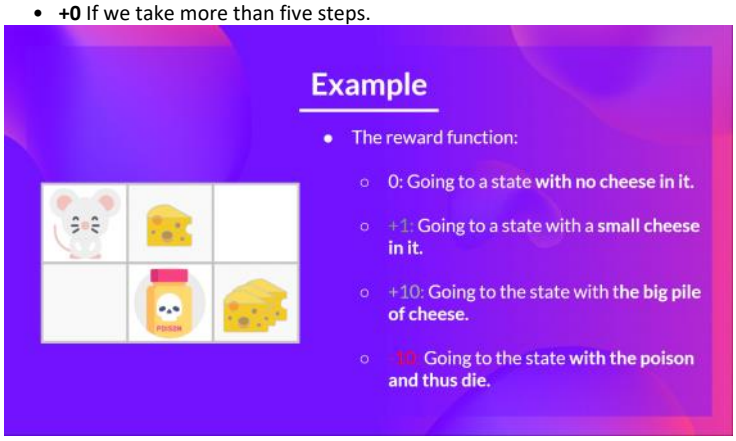
To better understand Q-Learning, let's take a simple example:

- You’re a mouse in this tiny maze. You always **start at the same starting point.**
- The goal is **to eat the big pile of cheese at the bottom right-hand corner** and avoid the poison. After all, who doesn’t like cheese?
- The episode ends if we eat the poison, **eat the big pile of cheese**, or if we take more than five steps.
- The learning rate is 0.1
- The discount rate (gamma) is 0.99



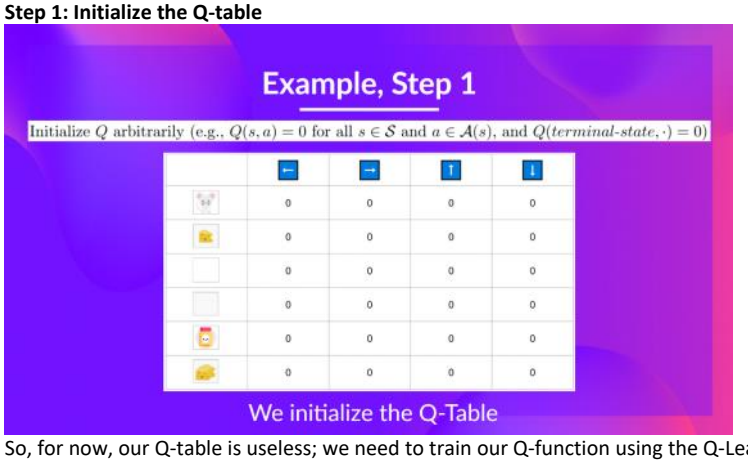
The reward function goes like this:

- **+0:** Going to a state with no cheese in it.
- **+1:** Going to a state with a small cheese in it.
- **+10:** Going to the state with the big pile of cheese.
- **-10:** Going to the state with the poison and thus dying.
- **+0** If we take more than five steps.



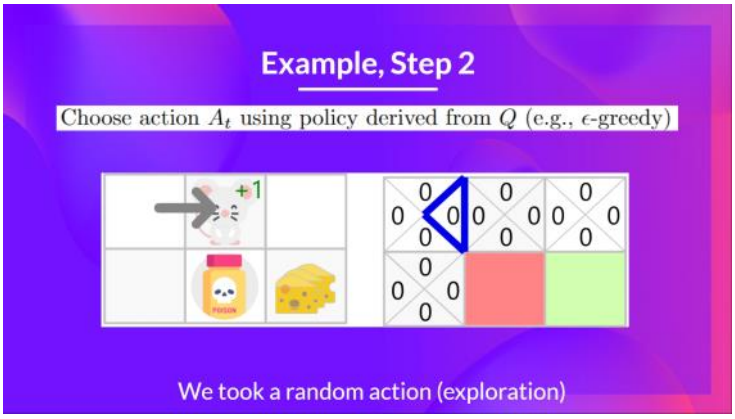
To train our agent to have an optimal policy (so a policy that goes right, right, down), we will use the Q-Learning algorithm.

a. **Step 1: Initialize the Q-table**

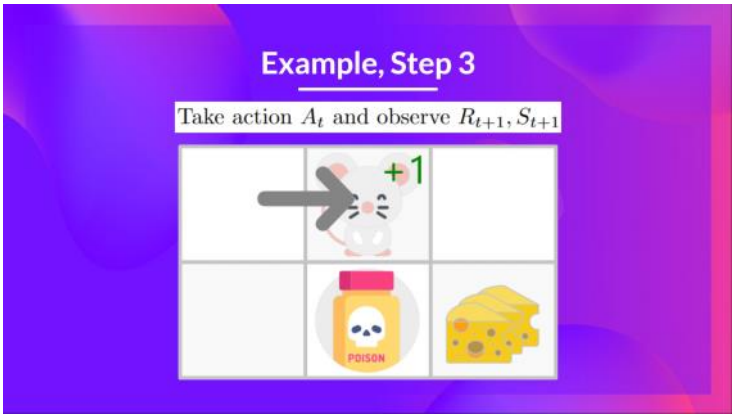


So, for now, our Q-table is useless; we need to train our Q-function using the Q-Learning algorithm.

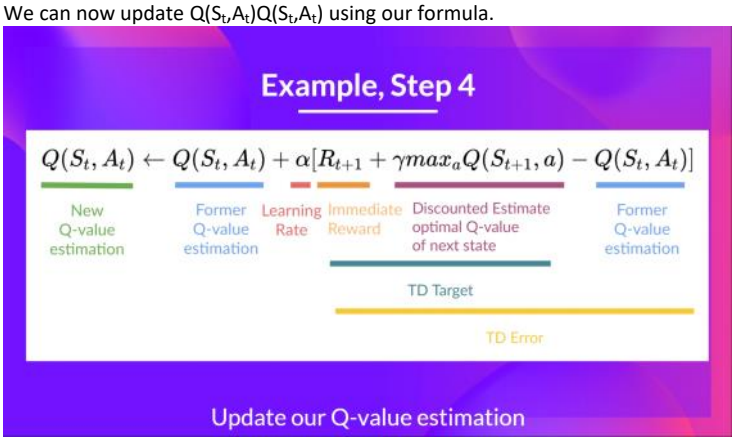
b. Training Timestep
   1. Step 2: Choose an action using the Epsilon Greedy Strategy
      Because epsilon is big (= 1.0), I take a random action. In this case, I go right.

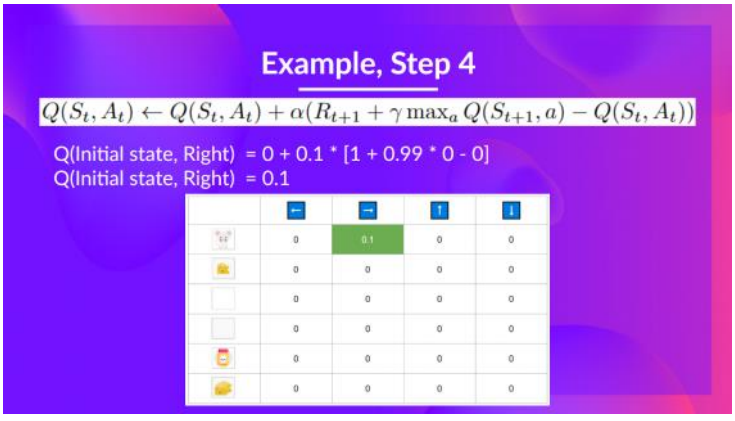


2. Step 3: Perform action At, get R_{t+1} and S_{t+1}
   By going right, I get a small cheese, so R_{t+1}=1and I'm in a new state.

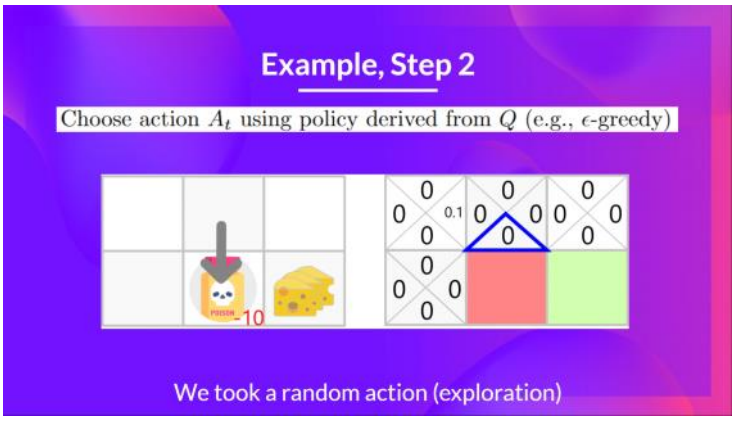


3. Step 4: Update Q(St, At)
   We can now update $Q(S_t,A_t)Q(S_t,A_t)$ using our formula.

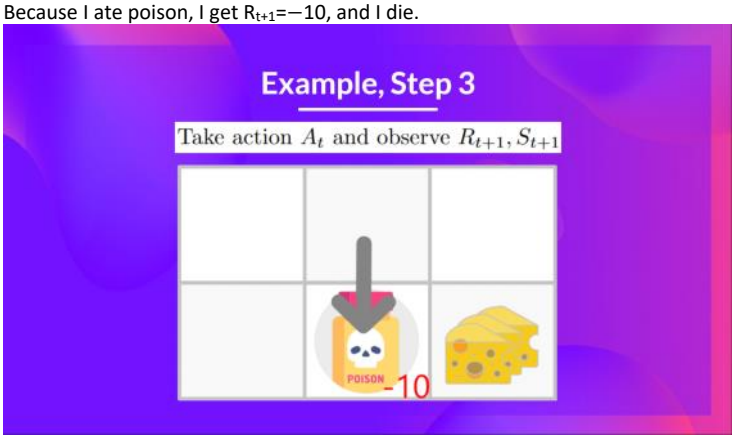




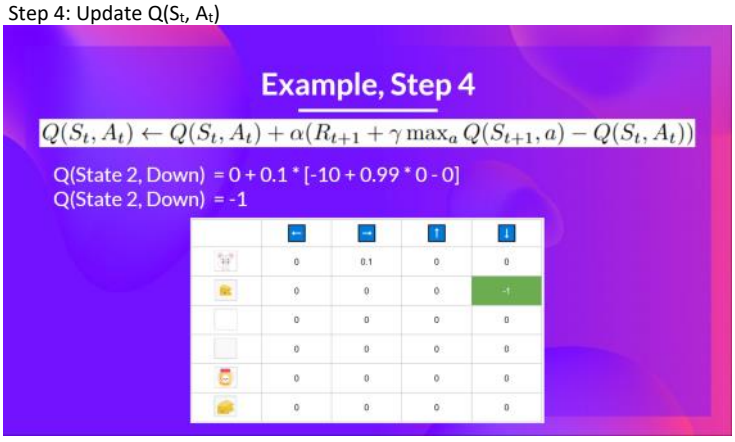
c. Timestep 2
   1. Step 2: Choose an action using the Epsilon Greedy Strategy
      I take a random action again, since epsilon=0.99 is big. (Notice we decay epsilon a little bit because, as the training progress, we want less and less exploration). I took the action 'down'. This is not a good action since it leads me to the poison.



2. Step 3: Perform action A_t, get R_{t+1} and S_{t+1}
   Because I ate poison, I get R_{t+1}=−10, and I die.



3. Step 4: Update $Q(S_t, A_t)$



Because we're dead, we start a new episode. But what we see here is that, with two explorations steps, my agent became smarter. As we continue exploring and exploiting the environment and updating Q-values using the TD target, the Q-table will give us a better and better approximation. At the end of the training, we'll get an estimate of the optimal Q-function.

11. The Difference Between policies
    a. Epsilon-Greedy Policy:
       a) The epsilon-greedy policy strikes a balance between exploration and exploitation. It works as follows:
       b) With probability (1 - \epsilon), the agent chooses the action associated with the highest estimated value (exploitation).
       c) With probability (\epsilon), the agent selects a random action (exploration).
       d) The goal is to explore new actions while still favoring the current best action.
       e) However, epsilon-greedy policies uniformly select random actions, which may not be optimal.
    b. Greedy Policy:
       a) The greedy policy is straightforward: always choose the action with the highest estimated value.
       b) It purely exploits the current best action based on the agent's knowledge.
       c) The drawback is that it lacks exploration. By always selecting the best-known action, it may miss out on discovering better alternatives.
    c. Soft Policies (e.g., Softmax):
       a) Soft policies address the limitations of pure exploration or exploitation.

b) The softmax policy assigns probabilities to actions based on their Q-values (estimated values).

c) Instead of blindly choosing the best action, it considers the relative quality of all actions.

d) Soft policies are often used in policy gradient methods and actor-critic architectures.